

4.2 Build the readings screens with TanStack Query

 Now we have started the fourth and final week of this project. We will be creating a front end for this product. Then once

Learning objective

What does TanStack Query do for you? Caching, loading and error states, refetching, retries

TanStack Query helps manage data coming from an API. It automatically handles **caching**, so previously loaded data can be reused, as well as **loading and error states** so the interface can show useful feedback while a request is running or if it fails. It can also **refetch** data when fresh information is needed and can retry failed requests, which reduces the amount of request-handling code I need to write myself.

What's the difference between a query and a mutation?

A **query** is mainly used when I want to **read or retrieve data**, such as using GET to view all readings for a branch or find one reading by UUID. A **mutation** is used when I want to **change data**, such as POST to create a reading, PATCH to update one, or DELETE to remove one. In my project, Page 3 mainly uses queries, while Page 2 mainly uses mutations.

Why do you need to invalidate a query after changing data, and what happens if you forget?

After I create, update or delete a reading, the data stored in the backend has changed, but TanStack Query may still have the old data saved in its cache. **Invalidating the query** tells TanStack Query that the cached data is now out of date and should be refreshed when needed. If I forget to invalidate it, the user could continue seeing old information, such as a deleted reading still appearing in the table or a newly created reading not appearing.

What does a good loading state and a good error state look like from the user's point of view?

A good **loading state** clearly tells the user that something is happening, for example showing "Loading temperature readings..." or changing a button to "Saving Reading...", rather than leaving the page looking frozen. A good **error state** should clearly explain that the operation failed and, where possible, tell the user why, such as an invalid temperature range or a reading not being found. This gives the user useful feedback and helps them understand what they need to do next.

Task



Build the three screens from the brief:

Screen 1 — Readings list. Branch selector at the top. Table of readings for that branch, newest first: date and time, fridge, min °C, max °C, status, recorded by. Excursion rows clearly flagged — and not by colour alone, because that fails anyone who can't distinguish it. Use a label too.

Screen 2 — Reading detail. Every field. If it's an excursion, say so prominently and show when the alert was raised.

Screen 3 — New reading form. Every field is spelled out in the brief with its type, whether it's required, its default and its validation message. Follow it. On a successful save, go back to the list and make sure the new reading is there without a manual refresh — that's the invalidation. If the reading was an excursion, tell the user an alert has been raised.

Handle every state properly

- Loading — while readings are being fetched
- Error — the API is down or returned a 500
- Empty — "No readings recorded for this branch yet". This is a real state, not an edge case. A branch that has never logged a reading is exactly the branch you'd want to notice
- Validation errors from the API — show the user what's actually wrong, not "something went wrong"

We are now creating the actual projects frontend now. 4.1 was just helping me get used to the frontend and setting it up now we are actually creating the frontend.

However, I want to add my changes that I made to the project to also be displayed in the frontend too. this will included.

- the DELETE and PATCH because my project uses REST APIs
- have the alert column
- The UUID being operated in the table
- And anything else that the backend does that I haven't listed (pretty sure I have listed everything)
- (making sure you are also doing the tings the ticket has asked)

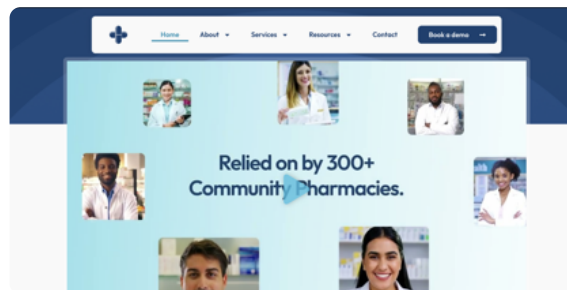
So the way I actually want this too be created is to have the 3 screens however, the first screen is just simple Home Screen that has information on it and about the encon company and more importantly how to uses the sight and THE MOST important part being navigation to the other two pages.

The screenshot shows the AWS IAM console with a list of tables. The table 'test1' is highlighted. The table has a partition key of 'id' and a sort key of 'date'. It is located in the 'us-east-1' region and has a size of 110 bytes. The table is in the 'Standard' table class.

Name	Status	Partition key	Sort key	Indexes	Replication Regions	Retention protection	Permissions	Read capacity mode	Write capacity mode	Total size	Table class
4-week-usage-hierarchy-state-lock	Active	uuid (S)	-	0 0	-	Off	-	On-demand	On-demand	110 bytes	Standard
All Users Table	Active	uuid (S)	-	0 0	-	Off	-	On-demand	On-demand	162 bytes	Standard
Alerts Table	Active	messageid (S)	-	0 0	-	Off	-	On-demand	On-demand	122 bytes	Standard
CrossDomainTableForPhone	Active	uuid (S)	-	0 0	-	Off	-	On-demand	On-demand	53 bytes	Standard
test1	Active	id (S)	date (S)	0 0	-	Off	-	On-demand	On-demand	110 bytes	Standard
user_subchain_database	Active	branchid (S)	readingid (S)	0 0	-	Off	-	On-demand	On-demand	1.6 kilobytes	Standard
user_subchain-readings-terraform	Active	readingid (S)	-	1 0	-	Off	-	On-demand	On-demand	0 bytes	Standard

- page 2 will be where the user can log and do the things with the readings; this is where a reading is made they can delete readings and edit readings. (GET is done in page three where the client will select a filter at the top for idivil reading or a whole branch readings) and when they input a reading and the Min or Max temp causes an excursion the alert should come up in a notification box I want it to look like the image I have put of the AWS DynamoDB table notification.
- page 3 is the list of readings doing what the ticket is saying (Branch selector at the top. Table of readings for that branch, newest first: date and time, fridge, min °C, max °C, status, recorded by. Excursion rows clearly flagged — and not by colour alone, because that fails anyone who can't distinguish it. Use a label too.) AND I want it to be displayed with all the values the dynamdb table has you remember with the excursion alert row, the UUID row separated from the date/time as well. It should be displayed in a table/database formatted the art column setters should be highlighted in red.

The design like colour scheme and text I want to look like the encon website so styles similar colours font spacing etc. like the image shown below is from the encon website.



Development

Understanding the actual structure the project will take

Understanding the structure the frontend will take ←

READ THIS

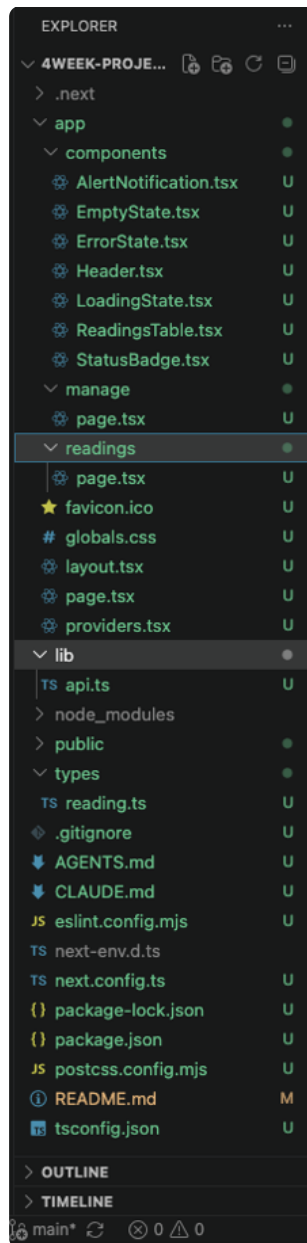
File format

First we are creating the new file format for the project it is shown bellow in both a write way and a image from my actual VS code.

New final structure

This is the structure I want us to build toward:

```
1 4week-project-frontend/
2
3 └─ app/
4     │
5     └─ components/
6         │
7         ├── Header.tsx
8         ├── StatusBadge.tsx
9         ├── AlertNotification.tsx
10        ├── ReadingsTable.tsx
11        ├── LoadingState.tsx
12        ├── ErrorState.tsx
13        └── EmptyState.tsx
14
15    └─ manage/
16        └─ page.tsx
17
18    └─ readings/
19        └─ page.tsx
20
21    └─ favicon.ico
22    └─ globals.css
23    └─ layout.tsx
24    └─ page.tsx
25    └─ providers.tsx
26
27 └─ lib/
28     └─ api.ts
29
30 └─ types/
31     └─ reading.ts
32
33 └─ public/
34
35 └─ .env.local
36 └─ .gitignore
37 └─ eslint.config.mjs
38 └─ next-env.d.ts
39 └─ next.config.ts
40 └─ package-lock.json
41 └─ package.json
42 └─ postcss.config.mjs
43 └─ README.md
44 └─ tsconfig.json
```



What each part in the file format does:

app/page.tsx

Your **Home page:**

1 /

It will contain:

- Encon-style design
- Cold Chain Operations Portal introduction
- Explanation of how the system works
- Navigation to Manage Readings
- Navigation to View Readings

```
app/manage/page.tsx
```

Your **Create / Edit / Delete** page:

```
1 | /manage
```

This handles:

```
1 | POST    → Create
2 | PATCH   → Edit
3 | DELETE  → Delete
```

```
app/readings/page.tsx
```

Your database-style reading page:

```
1 | /readings
```

This handles:

```
1 | GET whole branch
2 | GET individual reading
```

and shows:

```
1 | UUID
2 | Branch
3 | Date
4 | Time
5 | Min °C
6 | Max °C
7 | Status
8 | Alert Raised At
9 | Recorded By
```

Components

These are pieces used by the pages.

```
Header.tsx
```

The Encon-style navigation:

```
1 | [Logo]    Home    Manage Readings    View Readings
```

Because it will be shared by all three pages, we shouldn't duplicate it three times.

```
StatusBadge.tsx
```

Displays:

```
1 | ✓ OK
```

or:

```
1 | △ EXCURSION
```

This helps us meet the accessibility requirement because we're not relying only on colour.

`AlertNotification.tsx`

This is the notification box you specifically asked for.

Something like the AWS notification bar:

```
1 |  
2 | △ Excursion detected  
3 | Reading saved. An alert has been raised.  
4 |
```

`ReadingsTable.tsx`

The big DynamoDB/database-style table on Page 3.

`LoadingState.tsx`

Shown while TanStack Query is fetching.

`ErrorState.tsx`

Shown when the API fails.

`EmptyState.tsx`

Displays:

```
1 | No readings recorded for this branch yet.
```

`types/reading.ts`

This is important.

It will define what one backend reading looks like:

```
1 | export type Reading = {  
2 |   readingId: string;  
3 |   branchId: string;  
4 |   recordedAt: string;  
5 |   minTempC: number;  
6 |   maxTempC: number;  
7 |   status: "ok" | "excursion";  
8 |   recordedBy: string;  
9 |   alertRaisedAt?: string;  
10| };
```

Then the entire frontend uses the same TypeScript definition.

lib/api.ts

This will contain the actual communication with AWS.

Instead of scattering:

```
1 fetch(...)
```

everywhere, we'll eventually have functions such as:

```
1 getBranchReadings()
2 getReading()
3 createReading()
4 updateReading()
5 deleteReading()
```

So:

```
1 Pages
2   ↓
3 TanStack Query
4   ↓
5 api.ts
6   ↓
7 API Gateway
8   ↓
9 Lambda
```

providers.tsx

This is the TanStack Query setup we were just discussing.

It makes:

```
1 useQuery
2 useMutation
3 query caching
4 invalidation
```

available throughout the frontend.

.env.local

This will eventually contain things like the API URL.

For example:

```
1 NEXT_PUBLIC_API_BASE_URL=...
```

We will create it later when we actually connect AWS.

Do **not** put values in it yet.

 **NOTE: Why / How we structure Next.js projects**

Begin coding the front end (section one the start)

To start this frontend part of the project off I connected my Next.js frontend to the real AWS backend I built during Week 3.

I installed **TanStack Query** and created `providers.tsx` so the application could use query caching, loading states, error handling and later mutations. I updated `layout.tsx` to wrap the application in the TanStack Query provider, created a shared TypeScript `Reading` type in `types/reading.ts`, and created `lib/api.ts` to contain the functions that communicate with my REST API. I initially tried calling API Gateway directly from the browser using the API URL and API key, but the browser sent a CORS preflight `OPTIONS` request which API Gateway rejected with a `403`.

So rather than changing my working Week 3 backend, I changed the design so the browser now calls a **Next.js Route Handler**, which acts as a server-side proxy to API Gateway. I moved the API URL and API key to server-only environment variables in `.env.local` (This `.env.local` file you will not see in git hub because it was git ignored, due to it holding both my API URL and the API gateway key), meaning the API key is no longer exposed to browser-side JavaScript. I created `app/api/backend/[...path]/route.ts` to forward GET, POST, PATCH and DELETE requests to the real AWS API.

I also encountered a `Missing Authentication Token` error because the API base URL was initially incorrect; after correcting it so that it ended at the `/dev` stage rather than including the full readings path, the frontend successfully returned real DynamoDB readings. Finally, I used TanStack Query in `app/readings/page.tsx` to fetch the readings for `BRANCH-001`, displaying proper loading, error and empty states while initially showing the successful response as raw JSON to prove the frontend-to-backend connection worked before building the final table interface.

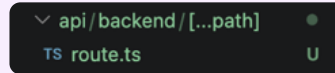
(Most of this section was getting the backend to connect with the frontend, there was basic design that I made to show that the backend values and process were now being displayed on the front end)

📌 **NOTE: What is a Next.js Route Handler??**

Route Handlers allow you to create custom request handlers for a given route using the

Web Request and Response APIs.

This Router Handler is defined in the router.ts/.js file inside the `app` directory. You can see my route file from the image below



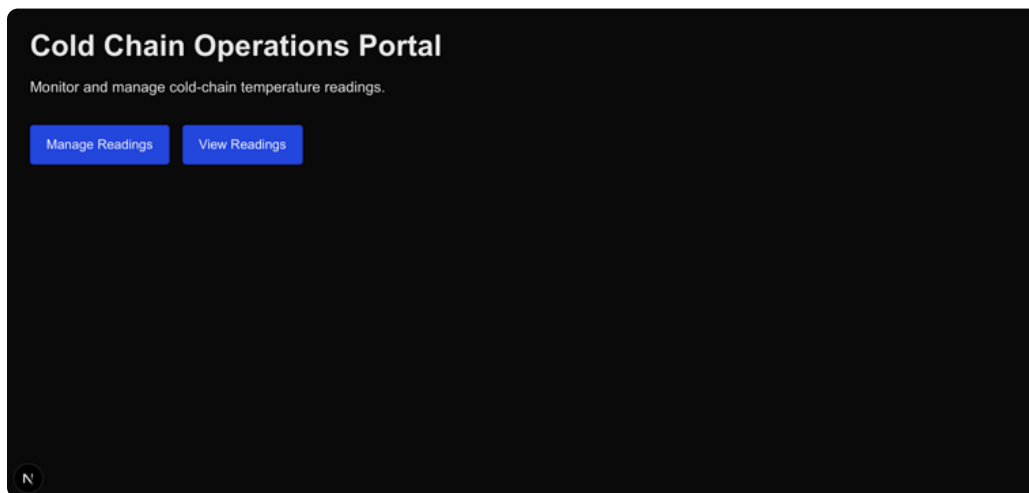
NOTE: What is a server-side proxy

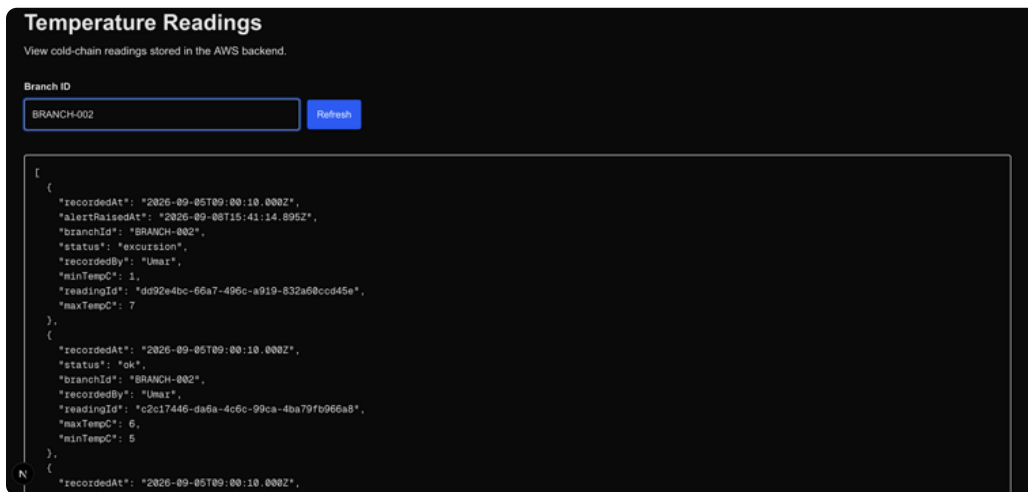
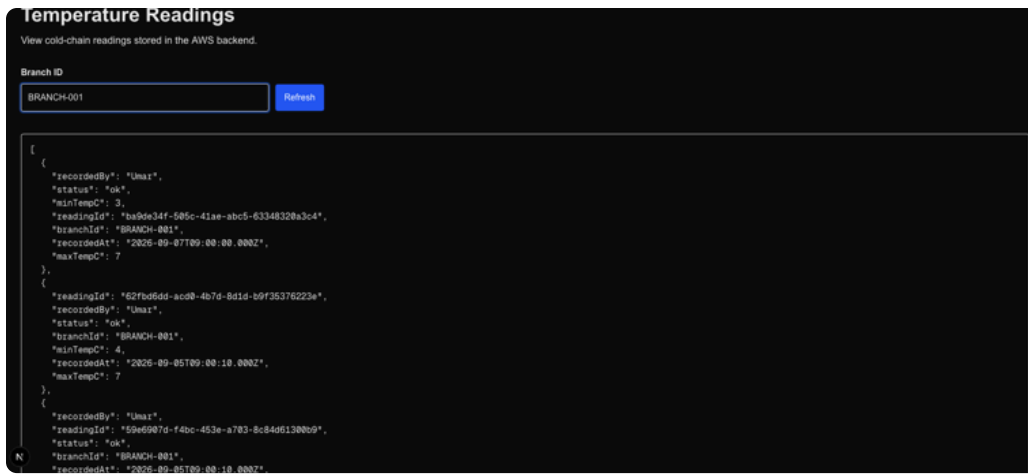
A server side proxy is an intermediary server that sits in front of backend web servers, intercepting incoming requests from clients before they reach the actual application

Understanding the whole development that we have done so far (Block by block / file by file).

[First development changes understanding code block by block](#) ← **READ THIS**

Result:





⚠️ IMPORTANT: 🌟

This will be the most difficult part I faced in the whole frontend section, because getting the backend to connect with the frontend is the most important part of this action of the project. Because the backend holds all the values and runs all the process like; create a reading, delete a reading, get a reading. Whereas the frontend just displays it in an interactive way for the client to use.

Therefore, now that I have completed link/configuring the backend to the frontend the most difficult part of this section is over and done with.

Coding the frontend (section two the start)

Next we are replacing the raw Jason output and the basic design of the view readings page with a clean layout. So we are going to be working in the components folder now. The raw JSON should disappear and be replaced by the first proper version of the database-style readings table, using the real backend data coming from DynamoDB. (This is basically the GET a whole branch function).

Result:

Temperature Readings

View cold-chain readings stored in the AWS backend.

Branch ID

BRANCH-001

Refresh

Reading UUID	Branch	Date	Time	Min °C	Max °C	Status	Alert Raised At	Recorded By
ba9de34f-585c-41aa-abc5-63348328a3c4	BRANCH-001	07/09/2026	10:00:00	3°C	7°C	✓ OK	—	Umar
627bd6d5-acd8-4b7d-8d1d-b9f36376223e	BRANCH-001	05/09/2026	10:00:10	4°C	7°C	✓ OK	—	Umar
59e987d-f4bc-453a-a783-8c4d61388b9	BRANCH-001	05/09/2026	10:00:10	6°C	7°C	✓ OK	—	Umar
8f21387b-3ee2-4be1-8531-aa78278faded	BRANCH-001	05/09/2026	10:00:10	1°C	7°C	⚠ EXCURSION	08/09/2026, 15:51:08	Umar
13842379-c68a-45bd-aad8-ad13c089ea38	BRANCH-001	05/09/2026	10:00:00	4°C	12°C	⚠ EXCURSION	08/09/2026, 15:50:30	Umar
87dc773e-fe6e-48ad-a6d4-c8bd0589268f	BRANCH-001	05/09/2026	10:00:00	4°C	8°C	✓ OK	—	Umar

Temperature Readings

View cold-chain readings stored in the AWS backend.

Branch ID

BRANCH-002

Refresh

Reading UUID	Branch	Date	Time	Min °C	Max °C	Status	Alert Raised At	Recorded By
a992e4bc-66a7-496c-a919-832a8cc045e	BRANCH-002	05/09/2026	10:00:10	1°C	7°C	⚠ EXCURSION	08/09/2026, 16:41:14	Umar
c2c1744e-da6a-4c6c-99ca-4ba79f996ea8	BRANCH-002	05/09/2026	10:00:10	5°C	6°C	✓ OK	—	Umar
8ff8d949-a73a-4578-9d9b-43a2ae7ef7ea	BRANCH-002	05/09/2026	10:00:10	-1°C	6°C	⚠ EXCURSION	09/09/2026, 09:27:02	Umar
3e6622c5-a839-4187-9487-a8194f31c2af	BRANCH-002	05/09/2026	10:00:10	1°C	12°C	⚠ EXCURSION	09/09/2026, 09:26:40	Umar

Now we are going to be working on the selector that will allow the client to GET only select readings at a time. (This is basically the GET one reading function) via the readings UUID. I have too utilise the UUID beacsue it is the way I was calling specific readings with postman and the way I code my backend too work.)

Result:

Temperature Readings

View branch readings or search for an individual reading using its UUID.

View

Branch Readings

Branch ID

BRANCH-001

Refresh

Reading UUID	Branch	Date	Time	Min °C	Max °C	Status	Alert Raised At	Recorded By
ba9de34f-585c-41ae-ab05-63348328a3c4	BRANCH-001	07/09/2026	10:00:00	3°C	7°C	✓ OK	---	Umar
6279e688-ac08-4b7d-8d16-1d9f25376223e	BRANCH-001	05/09/2026	10:00:10	4°C	7°C	✓ OK	---	Umar
59e0987d-f4bc-453e-a783-8c84d51388e9	BRANCH-001	05/09/2026	10:00:10	6°C	7°C	✓ OK	---	Umar
8f2138f2-3ce2-4be1-8531-a978278faded	BRANCH-001	05/09/2026	10:00:10	1°C	7°C	⚠ EXCURSION	08/09/2026, 15:51:08	Umar
13842379-c08a-45bd-aa28-4d13c889ba38	BRANCH-001	05/09/2026	10:00:00	4°C	12°C	⚠ EXCURSION	08/09/2026, 15:50:30	Umar

Temperature Readings

View branch readings or search for an individual reading using its UUID.

View

✓ Branch Readings

Individual Reading

Branch ID

BRANCH-001

Refresh

Reading UUID	Branch	Date	Time	Min °C	Max °C	Status	Alert Raised At	Recorded By
ba9de34f-585c-41ae-ab05-63348328a3c4	BRANCH-001	07/09/2026	10:00:00	3°C	7°C	✓ OK	---	Umar
6279e688-ac08-4b7d-8d16-1d9f25376223e	BRANCH-001	05/09/2026	10:00:10	4°C	7°C	✓ OK	---	Umar
59e0987d-f4bc-453e-a783-8c84d51388e9	BRANCH-001	05/09/2026	10:00:10	6°C	7°C	✓ OK	---	Umar
8f2138f2-3ce2-4be1-8531-a978278faded	BRANCH-001	05/09/2026	10:00:10	1°C	7°C	⚠ EXCURSION	08/09/2026, 15:51:08	Umar
13842379-c08a-45bd-aa28-4d13c889ba38	BRANCH-001	05/09/2026	10:00:00	4°C	12°C	⚠ EXCURSION	08/09/2026, 15:50:30	Umar

Temperature Readings

View branch readings or search for an individual reading using its UUID.

View

Individual Reading

Branch ID

BRANCH-001

Reading UUID

Enter reading UUID

Find Reading

Temperature Readings

View branch readings or search for an individual reading using its UUID.

View

Individual Reading

Branch ID

BRANCH-001

Reading UUID

ba9da34f-585c-42ae-8bc5-63348328a3c4

Find Reading

Reading Details

Reading UUID

ba9da34f-585c-42ae-8bc5-63348328a3c4

Recorded At

07/09/2026, 10:00:00

Minimum Temperature

3°C

Status

OK

Branch ID

BRANCH-001

Recorded By

Umar

Maximum Temperature

7°C

Alert Raised At

OK

Temperature Readings

View branch readings or search for an individual reading using its UUID.

View

Individual Reading

Branch ID

BRANCH-001

Reading UUID

82130fb-3ac2-4be1-8531-aa78270fada4

Find Reading

Reading Details

Excursion detected

This reading is outside the permitted temperature range.

Reading UUID

8f2338fb-3ac2-4be1-8531-aa78270fada4

Recorded At

05/09/2026, 10:00:10

Minimum Temperature

1°C

Status

EXCURSION

Branch ID

BRANCH-001

Recorded By

Umar

Maximum Temperature

7°C

Alert Raised At

08/09/2026, 15:51:08

EXCURSION

So page 3 the reading table page, does both these functions:

- **Branch Readings** → GET all readings for a branch
- **Individual Reading** → GET one reading using its UUID

Understanding the whole development that we have done so far (Block by block / file by file).

[Second development changes understanding code block by block](#) ←
READ THIS

Coding the frontend (section three the start)

The next major stage is **Page 2** /manage , where we will start using TanStack Query **mutations** for the first time:

POST → Create Reading

PATCH → Edit Reading

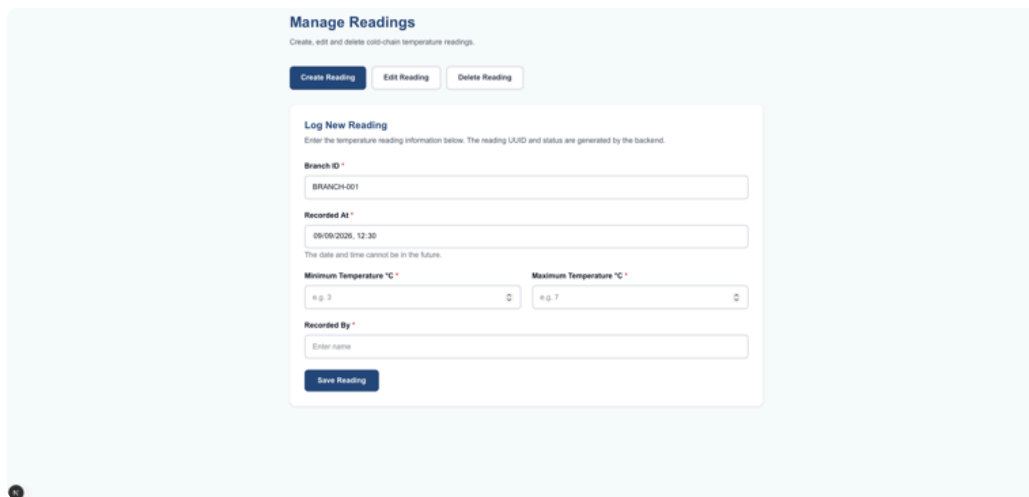
DELETE → Delete Reading

and then query invalidation will make the new/updated data automatically refresh on Page 3.

First I created the POST / create a reading

Result:

This is the crate reading screen:

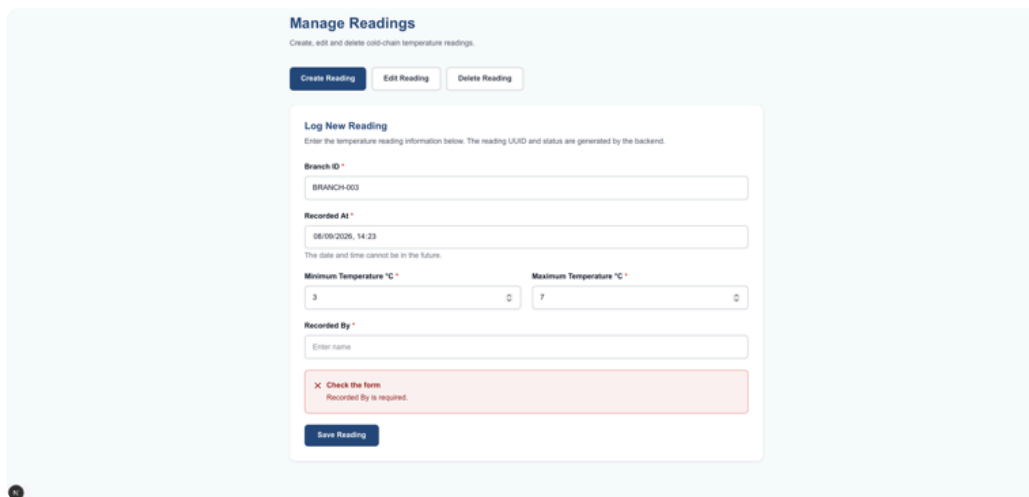


The screenshot shows a web application titled "Manage Readings" with the subtitle "Create, edit and delete cold-chain temperature readings." Below the title are three buttons: "Create Reading", "Edit Reading", and "Delete Reading". The main form is titled "Log New Reading" and contains the following fields:

- Branch ID ***: A text input field containing "BRANCH-001".
- Recorded At ***: A date and time input field showing "09/09/2026, 12:30". Below it, a small note says "The date and time cannot be in the future."
- Minimum Temperature °C ***: A numeric input field with "e.g. 3" and a spinner icon, containing the value "3".
- Maximum Temperature °C ***: A numeric input field with "e.g. 7" and a spinner icon, containing the value "7".
- Recorded By ***: A text input field with the placeholder "Enter name".

At the bottom of the form is a "Save Reading" button.

This is what happens when there is a columns value missing:



This screenshot shows the same "Log New Reading" form, but with a validation error. The fields are filled with "BRANCH-003", "09/09/2026, 14:23", "3", and "7". The "Recorded By" field is empty. A red error message box is displayed below the form, stating:

X Check the form
Recorded By is required.

The "Save Reading" button is still visible at the bottom.

Shows the reading been successfully:

Manage Readings
Create, edit and delete cold-chain temperature readings.

Create Reading Edit Reading Delete Reading

Log New Reading
Enter the temperature reading information below. The reading UUID and status are generated by the backend.

Branch ID *
BRANCH-003

Recorded At *
09/09/2026, 12:30
The date and time cannot be in the future.

Minimum Temperature °C *
e.g. 3

Maximum Temperature °C *
e.g. 7

Recorded By *
Enter name

✓ Reading saved
Reading 08fca5d2-826d-4625-8b3c-d69a079a3e2a was saved successfully.

Save Reading

This is what happens when a reading that is made has an excursion, it comes up with a warning sign tell the client there is an excursions.

Manage Readings
Create, edit and delete cold-chain temperature readings.

Create Reading Edit Reading Delete Reading

Log New Reading
Enter the temperature reading information below. The reading UUID and status are generated by the backend.

Branch ID *
BRANCH-003

Recorded At *
09/09/2026, 12:30
The date and time cannot be in the future.

Minimum Temperature °C *
e.g. 3

Maximum Temperature °C *
e.g. 7

Recorded By *
Enter name

⚠ Excursion detected
The reading was saved successfully and an excursion alert has been raised.

Save Reading

The readings are then update straight away into the Temperature reading page.

Temperature Readings
View branch readings or search for an individual reading using its UUID.

View
Branch Readings

Branch ID
BRANCH-003 Refresh

Reading UUID	Branch	Date	Time	Min °C	Max °C	Status	Alert Raised At	Recorded By
08fca5d2-826d-4625-8b3c-d69a079a3e2a	BRANCH-003	09/09/2026	14:23:00	3°C	7°C	✓ OK	—	Hussain
1599f18b-579d-435d-9283-527d0f8388e7	BRANCH-003	07/09/2026	14:25:00	-1°C	7°C	⚠ EXCURSION	09/09/2026, 14:25:36	Hussain

And they are also in the backend in AWS DynamoDB.

Table: umar-coldchain-readings-terraform - Items returned (13)

Scan started on September 09, 2026, 14:36:20

☐	readingId (String)	alertRaise...	branchId	maxTe...	minTe...	record...	record...	status
☐	87dcf79e-fe6e-48ad-ae...		BRANCH-001	8	4	2026-09-...	Umar	ok
☐	8f2130fb-3ce2-4be1-85...	2026-09-08...	BRANCH-001	7	1	2026-09-...	Umar	excursion
☐	1599f18b-579d-415d-9...	2026-09-09...	BRANCH-003	7	-1	2026-09-...	Hussain	excursion
☐	13842379-c60a-45bd-a...	2026-09-08...	BRANCH-001	12	4	2026-09-...	Umar	excursion
☐	59e6907d-f4bc-455e-a...		BRANCH-001	7	6	2026-09-...	Umar	ok
☐	0d6ca5d2-82dd-4625-a...		BRANCH-003	7	3	2026-09-...	Hussain	ok
☐	62fbd6dd-acd0-4b7d-8...		BRANCH-001	7	4	2026-09-...	Umar	ok
☐	3bbb22c5-a839-4107-9...	2026-09-09...	BRANCH-002	12	1	2026-09-...	Umar	excursion
☐	ba9de34f-505c-41ae-ab...		BRANCH-001	7	3	2026-09-...	Umar	ok
☐	8ff84949-a73a-4578-9b...	2026-09-09...	BRANCH-002	6	-1	2026-09-...	Umar	excursion
☐	c2c1744b-da6a-4c6c-99...		BRANCH-002	6	5	2026-09-...	Umar	ok
☐	3d67eb0b-cdd9-4b78-b...		BRANCH-001	7	3	2026-09-...	Hussain	ok
☐	dd92e4bc-66a7-496c-a...	2026-09-08...	BRANCH-002	7	1	2026-09-...	Umar	excursion

Aswell the standard validation check are also inlance and also show pop up notification if there is a problem with teh data being inputted. As seen bellow.

Manage Readings

Create, edit and delete cold-chain temperature readings.

[Create Reading](#) [Edit Reading](#) [Delete Reading](#)

Log New Reading

Enter the temperature reading information below. The reading UUID and status are generated by the backend.

Branch ID *

BRANCH-003

Recorded At *

07/09/2026, 14:37

The date and time cannot be in the future.

Minimum Temperature °C *

7

Maximum Temperature °C *

3

Recorded By *

Hussain

X Check the form
Maximum temperature cannot be lower than minimum temperature.

[Save Reading](#)

Secondly I created the PATCH / edit a reading

Result:

Manage Readings

Create, edit and delete cold-chain temperature readings.

[Create Reading](#) [Edit Reading](#) [Delete Reading](#)

Edit Reading

Enter the branch and reading UUID, then enter only the fields that you want to update.

Branch ID *

BRANCH-001

Reading UUID *

Enter UUID

Fields to update

Leave a field blank if you do not want to change it.

Minimum Temperature °C

Leave blank to keep current value

Maximum Temperature °C

Leave blank to keep current value

Recorded By

Leave blank to keep current value

[Update Reading](#)

Just like in postman I need uses the UUID of the specific reading to edit it. I can edit the temps and who it is recorded by. it will also show an excretion if it is edited and the values change too that.

Manage Readings
Create, edit and delete cold-chain temperature readings.

Create Reading Edit Reading Delete Reading

Edit Reading
Enter the branch and reading UUID, then enter only the fields that you want to update.

Branch ID *
BRANCH-001

Reading UUID *
7654b6fc-8547-4d8c-8c5e-ba21d7589785 was saved successfully.

Fields to update
Leave a field blank if you do not want to change it.

Minimum Temperature °C
-5

Maximum Temperature °C
12

Recorded By
Hassan

Update Reading

Manage Readings
Create, edit and delete cold-chain temperature readings.

Create Reading Edit Reading Delete Reading

Edit Reading
Enter the branch and reading UUID, then enter only the fields that you want to update.

Branch ID *
BRANCH-001

Reading UUID *
7654b6fc-8547-4d8c-8c5e-ba21d7589785

Fields to update
Leave a field blank if you do not want to change it.

Minimum Temperature °C
Leave blank to keep current value

Maximum Temperature °C
Leave blank to keep current value

Recorded By
Leave blank to keep current value

⚠ Excursion detected
The reading was updated successfully and is currently marked as an excursion.

Update Reading

Manage Readings
Create, edit and delete cold-chain temperature readings.

Create Reading Edit Reading Delete Reading

Edit Reading
Enter the branch and reading UUID, then enter only the fields that you want to update.

Branch ID *
BRANCH-001

Reading UUID *
7654b6fc-8547-4d8c-8c5e-ba21d7589785

Fields to update
Leave a field blank if you do not want to change it.

Minimum Temperature °C
Leave blank to keep current value

Maximum Temperature °C
Leave blank to keep current value

Recorded By
Leave blank to keep current value

✓ Reading updated
Reading 7654b6fc-8547-4d8c-8c5e-ba21d7589785 was updated successfully.

Update Reading

The file can be seen edited in both the app's reading viewer and the actual backend DynamoDB.

Temperature Readings

View branch readings or search for an individual reading using its UUID

View

Branch Readings

Branch ID

BRANCH-001

Refresh

Reading UUID	Branch	Date	Time	Min °C	Max °C	Status	Alert Raised At	Recorded By
3d67e0b8-cd09-4d78-b088-95f13856e87	BRANCH-001	06/09/2026	14:15:00	3°C	7°C	✓ OK	---	Hussain
ba9de34f-585c-41aa-abc5-63348320a3c4	BRANCH-001	07/09/2026	10:00:00	3°C	7°C	✓ OK	---	Umar
76548bfc-8547-4d8c-8c5e-ba21d71689785	BRANCH-001	06/09/2026	14:54:00	3°C	7°C	✓ OK	09/09/2026, 14:56:27	Hussain
62fbd6dd-acd0-4b7d-8d1d-10f735376223a	BRANCH-001	05/09/2026	10:00:10	4°C	7°C	✓ OK	---	Umar
59e6907d-f4bc-453e-a782-8c849130809	BRANCH-001	05/09/2026	10:00:10	6°C	7°C	✓ OK	---	Umar
8721387b-3ca2-4be1-8531-aa782787a08d	BRANCH-001	05/09/2026	10:00:10	1°C	7°C	⚠ EXCURSION	08/09/2026, 15:51:08	Umar
13842379-c08a-45bd-aa29-4d13c909ba38	BRANCH-001	05/09/2026	10:00:00	4°C	12°C	⚠ EXCURSION	08/09/2026, 15:50:30	Umar
87dcf79e-fe5e-4bad-ae94-c0bd05492687	BRANCH-001	05/09/2026	10:00:00	4°C	8°C	✓ OK	---	Umar

Reading beginning with 76540bfc (beacsue I change the reading twice it says it is okay but it still logs that it had an alrt)

Table: umar-coldchain-readings-terraform - Items returned (14)

Scan started on September 09, 2026, 15:00:23

Actions

Create Item

checkbox	readingId (String)	alertRaisedAt	branch...	maxTe...	minTe...	record...	record...	status
☐	87dcf79e-fe5e-4bad-ae94...		BRANCH...	8	4	2026-09-...	Umar	ok
☐	8f2130fb-3ca2-4be1-8531...	2026-09-08T14:51:08.144Z	BRANCH...	7	1	2026-09-...	Umar	excursion
☐	1599f18b-579d-415d-920...	2026-09-09T13:25:36.820Z	BRANCH...	7	-1	2026-09-...	Hussain	excursion
☐	13842379-c08a-45bd-aa2...	2026-09-08T14:50:30.821Z	BRANCH...	12	4	2026-09-...	Umar	excursion
☐	59e6907d-f4bc-453e-a70...		BRANCH...	7	6	2026-09-...	Umar	ok
☐	76540bfc-8547-4d8c-8c5e...	2026-09-09T13:56:27.743Z	BRANCH...	7	3	2026-09-...	Hussain	ok
☐	0d6ca5d2-82dd-4625-ab3...		BRANCH...	7	3	2026-09-...	Hussain	ok
☐	62fbd6dd-acd0-4b7d-8d1...		BRANCH...	7	4	2026-09-...	Umar	ok
☐	3bbb22c5-a839-4107-948...	2026-09-09T08:26:40.811Z	BRANCH...	12	1	2026-09-...	Umar	excursion
☐	ba9de34f-505c-41ae-abc5...		BRANCH...	7	3	2026-09-...	Umar	ok
☐	8ff84949-a73a-4578-9b9...	2026-09-09T08:27:02.570Z	BRANCH...	6	-1	2026-09-...	Umar	excursion
☐	c2c17446-da6a-4c6c-99ca...		BRANCH...	6	5	2026-09-...	Umar	ok
☐	3d67e0b8-cd09-4b78-b80...		BRANCH...	7	3	2026-09-...	Hussain	ok
☐	dd92e4bc-66a7-496c-a91...	2026-09-08T15:41:14.895Z	BRANCH...	7	1	2026-09-...	Umar	excursion

Thirdly I have created the final function the DELETE / delete a reading function:

Result:

Manage Readings

Create, edit and delete cold-chain temperature readings.

Create Reading

Edit Reading

Delete Reading

Delete Reading

Enter the branch and UUID of the reading you want to permanently delete.

Branch ID *

BRANCH-001

Reading UUID *

8d6ca5d2-82dd-4625-ab3c-e69a879a3e2d

Delete Reading

The delete has to be confirmed

Manage Readings

Create, edit and delete cold-chain temperature readings.

Create Reading

Edit Reading

Delete Reading

Delete Reading

Enter the branch and UUID of the reading you want to permanently delete.

Branch ID *

BRANCH-001

Reading UUID *

8b6ca5d2-82d8-4625-a03c-d69a879a3e2a

Confirm deletion

Are you sure you want to permanently delete this reading?

8b6ca5d2-82d8-4625-a03c-d69a879a3e2a

Cancel

Confirm Delete

Still has data validation

Manage Readings

Create, edit and delete cold-chain temperature readings.

Create Reading

Edit Reading

Delete Reading

Delete Reading

Enter the branch and UUID of the reading you want to permanently delete.

Branch ID *

BRANCH-001

Reading UUID *

8b6ca5d2-82d8-4625-a03c-d69a879a3e2a

X Reading could not be deleted

Reading not found

Confirm deletion

Are you sure you want to permanently delete this reading?

8b6ca5d2-82d8-4625-a03c-d69a879a3e2a

Cancel

Confirm Delete

When everything is correct and the delete has been confirmed the system will delete it

Manage Readings

Create, edit and delete cold-chain temperature readings.

Create Reading

Edit Reading

Delete Reading

Delete Reading

Enter the branch and UUID of the reading you want to permanently delete.

Branch ID *

BRANCH-001

Reading UUID *

3d57eb80-cd89-4b78-b888-985f1b856e8f

X Reading could not be deleted

Request failed with status 500

Confirm deletion

Are you sure you want to permanently delete this reading?

3d57eb80-cd89-4b78-b888-985f1b856e8f

Cancel

Confirm Delete

Okay so we ran into this error above it shows a value of 500 when DELETE was sent through, however after running a postman request to see if that value would be delta it came up with a 404 not found. So I went into the database but the reading had actually been deleted and there was no errors in the CloudWatch. Therefore, the issue wasn't because the backend was failing what was happening was;

Frontend DELETE → AWS deletes the reading ✓ → AWS returns 204 No Content ✓ --
→ Next.js proxy tries to create a response with an empty body for status 204 ✗ → Next.js throws a 500

So I have inputted this block of code into the `app/api/backend/[...path]/route.ts`

```
1 if (response.status === 204) {  
2   return new Response(  
3     null,  
4     {  
5       status: 204,  
6     }  
7   );  
8 }
```

so this will mean 204 + empty body equals success however before 204 + full body would equal success. this code changes that.

NOW that it has been fixed I created a new reading and deleted it and everything worked successfully.

The screenshot shows a web application titled "Manage Readings" with the subtitle "Create, edit and delete cold-chain temperature readings." At the top, there are three buttons: "Create Reading", "Edit Reading", and "Delete Reading". Below these is a "Log New Reading" section with the instruction "Enter the temperature reading information below. The UUID and status are generated by the backend." The form contains the following fields: "Branch ID" with the value "BRANCH-001", "Recorded At" with the value "05/06/2020, 15:56", "Minimum Temperature °C" with the value "4", and "Maximum Temperature °C" with the value "7". There is also a "Recorded By" field with the value "Ali". A "Save Reading" button is located at the bottom of the form.

Manage Readings
Create, edit and delete cold-chain temperature readings.

Create Reading Edit Reading Delete Reading

Log New Reading
Enter the temperature reading information below. The UUID and status are generated by the backend.

Branch ID *
BRANCH-001

Recorded At *
09/08/2025, 12:30

Minimum Temperature °C *
Maximum Temperature °C *

Recorded By *

✓ Reading saved
Reading 072dfca2-3160-43ee-9425-0829eca1c0a4 was saved successfully.

Save Reading

Manage Readings
Create, edit and delete cold-chain temperature readings.

Create Reading Edit Reading Delete Reading

Delete Reading
Enter the branch and UUID of the reading you want to permanently delete.

Branch ID *
BRANCH-001

Reading UUID *
Enter UUID

✓ Reading deleted
Reading 072dfca2-3160-43ee-9425-0829eca1c0a4 was deleted successfully.

Delete Reading

Understanding the whole development that we have done so far (Block by block / file by file).

[Third development changes understanding code block by block](#) ← **READ THIS**

Coding the frontend (section three the start)

The next two things I am going to be creating is the navigation bar / header and the home pages proper design. And I am going to be reviewing the desgin and any bugs or things ibuprofen want changing I will implement.

First I implemented the header for navigation between the different pages

Result:

CC Cold Chain Operations Portal

HomeManage ReadingsView Readings

COLD CHAIN MONITORING

Cold Chain Operations Portal

A central portal for recording, monitoring and managing cold-chain temperature readings across branches.

Manage ReadingsView Readings

ABOUT THE PORTAL

Monitor temperature readings from one place

This operations portal connects directly to the AWS cold-chain backend and allows users to create, update, delete and retrieve temperature readings.

Every reading is given a unique UUID by the backend and is automatically checked to determine whether the recorded temperatures are within the permitted cold-chain range.

TEMPERATURE RULE

2°C to 8°C

A reading is considered safe when the minimum and maximum temperatures remain within the permitted range.

⚠ Excursion

If the minimum falls below 2°C or the maximum rises above 8°C, the reading is marked as an excursion and the alert workflow is triggered.

CC Cold Chain Operations Portal

HomeManage ReadingsView Readings

Manage Readings

Create, edit and delete cold-chain temperature readings.

Create ReadingEdit ReadingDelete Reading

Log New Reading

Enter the temperature-reading information below. The UUID and status are generated by the backend.

Branch ID *

BRANCH-001

Recorded At *

10/09/2026, 12:30

Minimum Temperature °C *

Maximum Temperature °C *

Recorded By *

Save Reading

CC Cold Chain Operations Portal

HomeManage ReadingsView Readings

Temperature Readings

View branch readings or search for an individual reading using its UUID.

View

Branch Readings

Branch ID

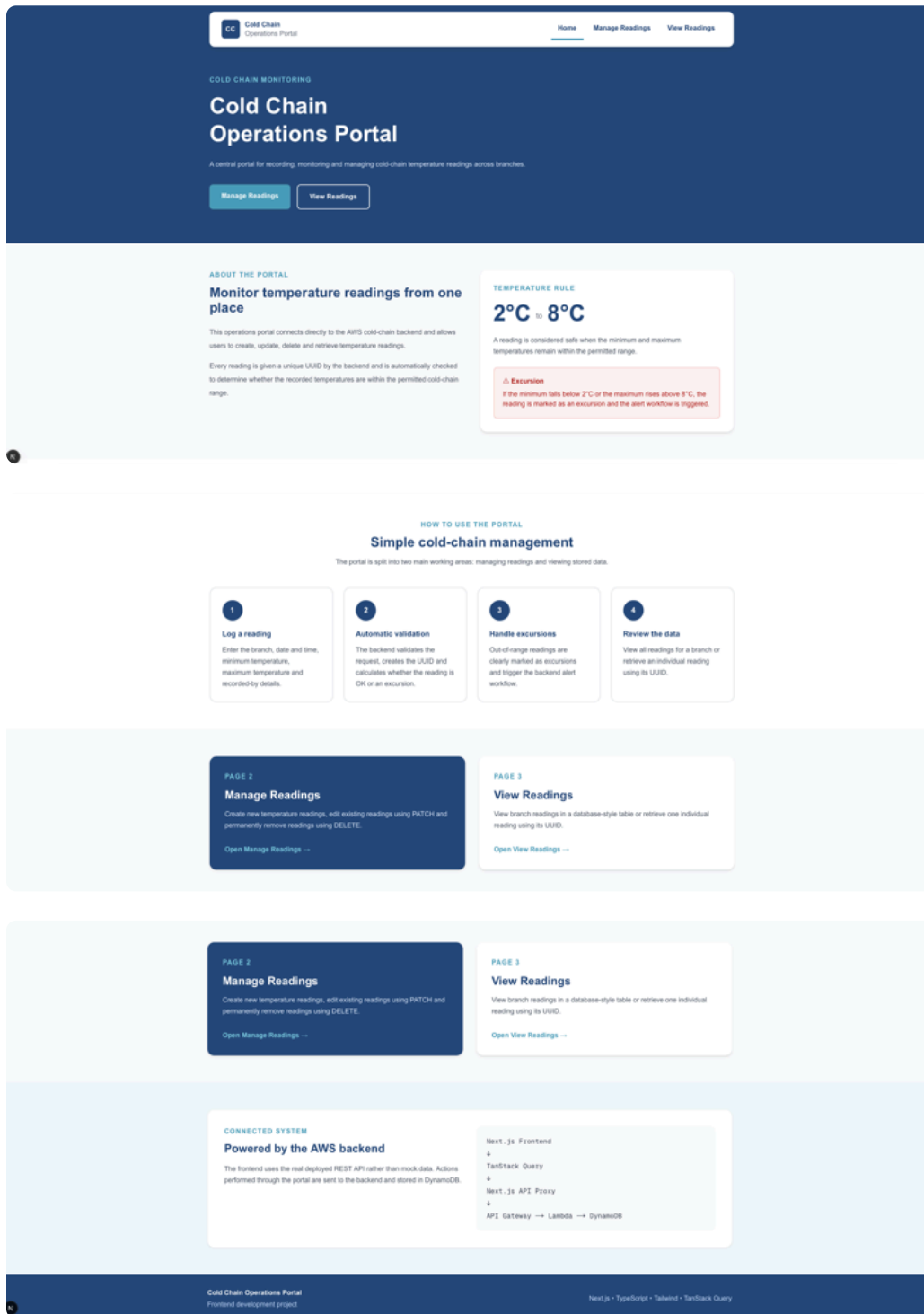
BRANCH-001

Refresh

Reading UUID	Branch	Date	Time	Min °C	Max °C	Status	Alert Raised At	Recorded By
76548bfc-8547-4d8c-8c5e-ba21d75b9785	BRANCH-001	09/09/2026	14:54:00	3°C	7°C	✓ OK	09/09/2026, 14:56:27	Hussain
627b6660-ac08-4b7d-b41d-b9f35376223e	BRANCH-001	09/09/2026	10:00:10	4°C	7°C	✓ OK	—	Umar
13ae0387d-f4bc-453e-a783-bc84a9138869	BRANCH-001	09/09/2026	10:00:10	6°C	7°C	✓ OK	—	Umar
8f2138fb-3ca2-4be1-8531-ba78278fa98d	BRANCH-001	09/09/2026	10:00:10	1°C	7°C	⚠ EXCURSION	08/09/2026, 15:51:08	Umar
13842379-c68a-45b6-aa28-4d13c909ba38	BRANCH-001	09/09/2026	10:00:00	4°C	12°C	⚠ EXCURSION	08/09/2026, 15:50:30	Umar
4b052ed3-5ea5-472b-a7fe-b9e6b6faa573	BRANCH-001	02/09/2026	18:24:00	1°C	2°C	⚠ EXCURSION	09/09/2026, 18:24:40	me

Then I created the Home Screen:

Result:



Okay now after re-viewing the Home Screen I changed certain things about the home screens design that I didn't like:

- The x button on the pop ups need to actually work none of them actually work I have clicked on them but they do not remove themselves. (You can see one off the pop ups in the

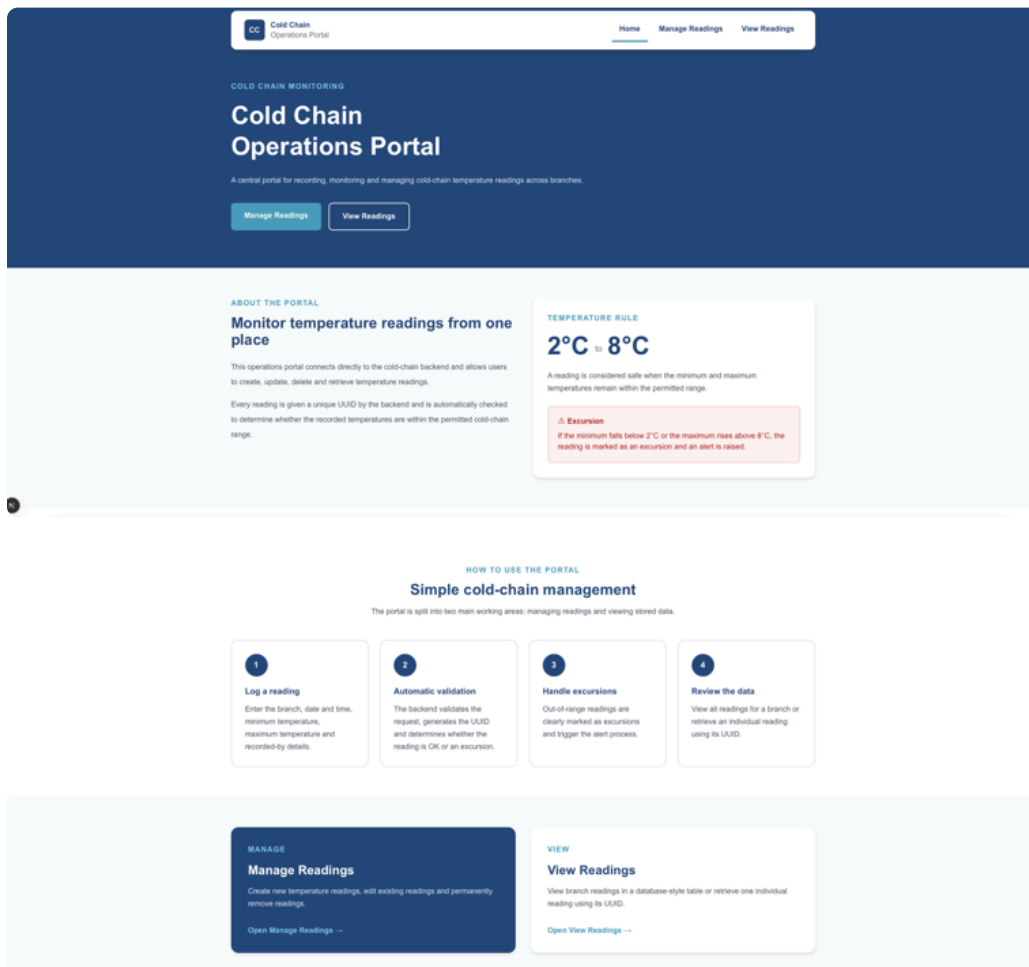
screenshot I have inputted)

- For the home page I want you too get ride off any off the writing that says AWS backend and just put backend we don't need to go too far into
- In the footer add my name Umar Asif Hussain and for where it say Next.js and the rest off the software that was used put developed with : ...
- The box at the bottom I don't want it to be talking about the backend, just have it talking about me. Saying how I am a junior developer at Encon Pharma and say that this is one off my projects I have completed through the duration of my internship

Result

The files we need to change to make these improvements are:

```
1 app/components/AlertNotification.tsx
2 app/manage/page.tsx
3 app/page.tsx
```





Understanding the whole development that we have done so far (Block by block / file by file).

[Fourth development changes understanding code block by block](#) ← **READ THIS**

 **Questions / Queries / Pushback**